Safe⟨T⟩: Automaton-Theoretic Runtime Monitoring of Blackbox Distributed Systems

Karuna Grewal, Cornell University, kgrewal@cs.cornell.edu
Umang Mathur, National University of Singapore, umathur@nus.edu.sg

Length: 3 hours

Abstract

Modern safety-critical systems are increasingly built from distributed blackbox components: cloud services, third-party APIs, autonomous systems, and AI agents whose source code or internal state may be unavailable for inspection. Ensuring their safety requires enforcing policies that constrain inter-component interactions, including multi-hop execution structure, data relationships, and concurrency. However, enforcing such expressive policies at runtime is challenging because the monitor needs to reason about distributed execution histories while being efficient and non-invasive. This creates a tension between policy expressiveness and practical deployability.

This tutorial presents Safe⟨T⟩, an automata-theoretic distributed runtime monitoring framework that bridges this gap. The framework includes a set of policy languages that get compiled into automaton-theoretic runtime monitors. We model distributed executions as structured words and use automata to incrementally summarize policy-relevant information: SafeTree uses nested words and visibly pushdown automata for hierarchical executions, SafeNom uses nominal words and automata for data-aware policies, and SafePar uses series-parallel nested words and series-parallel visibly pushdown automata for concurrent executions. We show how these theoretical foundations enable lightweight, distributed enforcement using the Istio service mesh and Envoy proxies, without modifying application code. Through worked examples and a hands-on demonstration, participants will learn the end-to-end pipeline from policy specification to automata-based enforcement. We conclude by examining a future extension of this technique to agentic AI systems.

The tutorial material is primarily based on our recent work [5, 8, 7, 6, 4] and on landmark results in automata theory and runtime verification [1, 2, 3, 10, 9, 11].

Biography

- **Karuna Grewal** is a Ph.D. candidate in Computer Science department at Cornell University. Her research focus is to design provably correct runtime monitoring mechanisms for enforcing safety and security properties in systems like cloud application and agentic AI systems without access to the source code. Her work includes the Safe⟨T⟩ [4] runtime monitoring framework, which has been recognized by SIGPLAN John Vlissides Award 2026. Her research is supported by a LinkedIn Ph.D. Fellowship. [Website]
- **Umang Mathur** is an Assistant Professor in the School of Computing at the National University of Singapore. His research interests include programming languages, formal methods, program analysis, hardware design and concurrency. His work has previously been recognized by ACM SIGPLAN Research Highlights and CACM Research Highlights. [Website]

Description

This tutorial examines how to specify and enforce expressive safety policy for distributed systems whose components must be treated as black boxes.

Topic and Focus

Distributed systems like cloud applications require reasoning about hierarchical interactions, data relationships, and concurrent executions. Consider a healthcare application where requests pass through multiple services. A safety policy may allow sensitive data to be sent to an external lab only after de-identification, while prohibiting calls from within a confidential clinical workflow. Checking the lab API call alone is insufficient: its safety depends on where it occurs in the workflow and how data has been processed across services. Similarly, concurrent operations may require exactly one successful write or a specific combination of outcomes. These policies require capturing execution structure and data relationships.

Enforcing such policies is challenging when the internal implementations of individual components are unavailable. However, their inter-component interactions remain observable at the network layer, providing a natural place to deploy non-invasive runtime monitors. To be practical, these monitors must operate efficiently and in a distributed manner, without maintaining the entire execution history centrally.

Our Safe⟨T⟩ monitoring framework, uses different automata to concisely summarize hierarchical interactions (SafeTree), data relationships (SafeNom), and concurrency (SafePar). For each, we introduce the

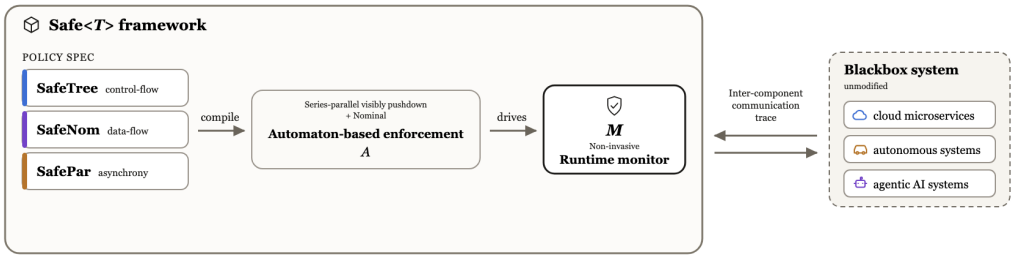


Figure 1: Overview of the Safe⟨T⟩ framework for distributed runtime monitoring of black-box systems.

underlying formal languages and automata. SafeTree uses nested words and visibly pushdown automata to track ancestor context. SafeNom uses nominal languages and automaton to enforce equality and freshness checks. SafePar uses series-parallel nested words and series–parallel visibly pushdown automata to combine branch outcomes.

A key consideration throughout the tutorial is that the choice of execution model and automaton affects the efficiency and deployability of the monitor. We then show the implementation of the automaton-based enforcement mechanism as an inline and non-invasive distributed monitoring framework, Safe⟨T⟩ (in fig 1), running atop a commonly used service mesh networking layer. Lastly, we highlight open research challenges and opportunities to extend these techniques to emerging domains, like guardrails for agentic AI systems.

Objective

The tutorial has the following objectives:

1. **Motivate expressive safety policies for black-box systems.** Show why real-world safety requirements depend on multi-hop execution structure and data relationships that cannot be captured by individual API-level checks.
2. **Identify the requirements for practical runtime enforcement.** Explain the challenges of monitoring distributed executions and the need for efficient, non-invasive, inline enforcement.
3. **Introduce automata-theoretic foundations for distributed monitoring.** Choose trace models that capture policy-relevant structure and data. Design automata that incrementally summarize the information needed for enforcement.
4. **Demonstrate the end-to-end monitoring pipeline.** Hands-on exercise of using the Safe⟨T⟩ monitor on an example safety policy to illustrate the policy → automaton → monitor pipeline.
5. **Discuss open problems.** Discuss open challenges in expressive runtime monitoring, including for emerging agentic AI systems. Motivate students and researchers to explore new applications of automata-theoretic techniques.

Tutorial Outline

The 3-hour tutorial is organized into four parts, with questions and interactive examples throughout.

Topic	Minutes	Content
I. Motivation	25	Black-box distributed systems; multi-hop safety policies; execution structure and data relationships.
II. Monitoring Requirements	20	Inline enforcement; non-invasive deployment; distributed monitoring; expressiveness vs. deployability.
III. Trace Models and Automata		
SafeTree	30	Structural policies, nested word, visibly pushdown automata, and incremental distributed monitoring.
SafeNom	30	Data-aware policies, nominal words, nominal automata, equality and freshness.
SafePar	30	fork-join concurrency policies, series-parallel nested words, series-parallel visibly pushdown automata.
IV. Hands-on Tutorial/QA	45	End-to-end policy → automaton → monitor pipeline; test a safety policy using Safe⟨T⟩; agentic security future directions.
Total	**180**	**3 hours**

Presentation Approach

The tutorial combines first-principles explanations with a running example and participant exercises.

In Part I, we will examine why a seemingly reasonable single-hop policy permits an unsafe multi-hop interaction.

In Part II, participants will identify what a monitor can observe at a component boundary and what information must accompany a request to make an inline decision.

In Part III, we will trace automaton transitions by hand and compare how the three models capture hierarchy, data, and concurrency. We introduce each automata-theoretic technique by first motivating the safety policies it is designed to enforce.

The final part will walk through a Safe⟨T⟩ policy from specification to enforcement. Participants will start with an example safety requirement, inspect its compiled automaton, and run permitted and violating traces against the distributed monitor to observe which interactions are blocked.

We plan to provide the tool, example policies, and setup instructions in advance, together with a prepared demonstration for attendees who do not run the environment locally. We will close with an interactive discussion of unresolved challenges.

Prerequisite Knowledge

The tutorial assumes familiarity with finite state automaton, pushdown automaton, and regular languages. No background in distributed systems and networks is required. Concepts about microservices, service mesh, nested word language models will be introduced from first principles.

Target Audience

The tutorial welcomes both graduate students and experienced researchers. The tutorial brings together automata theory, runtime verification, and distributed systems, making it relevant to several research communities represented at POPL, including researchers studying safety enforcement for AI agents.

1. **Automata theory and formal language researchers** will learn about a practical use of nested words [2], series-parallel nested words [8], and nominal words [3] for runtime monitoring,
2. **Runtime monitoring and dynamic analysis researchers** will learn about a decentralized monitoring architecture that does not require instrumenting the networked system.
3. **PL+{systems, security, AI safety} researchers** will learn about a complete monitoring framework: specification languages, compiler, and implemented runtime monitor.

Outreach

We plan to advertise it through programming languages and formal methods mailing lists (including TYPES and PLTea), seminar groups at Cornell, NUS, and UIUC, and relevant distributed systems communities.

Prior offerings

The tutorial has not been held previously. It is primarily based on the Safe⟨T⟩ project [6, 5, 7, 8] and foundational work on visibly pushdown and nested-word automata [1, 2]. Slides, illustrative policy examples, and tool instructions will be made available to participants.

References

[1] Rajeev Alur and P. Madhusudan. 2004. Visibly pushdown languages. In *Proceedings of the 36th Annual ACM Symposium on Theory of Computing, Chicago, IL, USA, June 13-16, 2004*, László Babai (Ed.). ACM, 202–211. doi:10.1145/1007352.1007390

[2] Rajeev Alur and P. Madhusudan. 2006. Adding Nesting Structure to Words. In *Developments in Language Theory, 10th International Conference, DLT 2006, Santa Barbara, CA, USA, June 26-29, 2006, Proceedings (Lecture Notes in Computer Science, Vol. 4036)*, Oscar H. Ibarra and Zhe Dang (Eds.). Springer, 1–13. doi:10.1007/11779148_1

[3] Mikolaj Bojanczyk, Bartek Klin, and Slawomir Lasota. 2014. Automata theory in nominal sets. *Log. Methods Comput. Sci.* 10, 3 (2014). doi:10.2168/LMCS-10(3:4)2014

[4] Karuna Grewal. 2026. Safe<T>: Automaton-Theoretic Runtime Monitoring of Blackbox Distributed Systems. In *Companion Proceedings of the 2026 ACM SIGPLAN International Conference on Systems, Programming, Languages, and Applications: Software for Humanity* (Oakland, CA, USA) *(SPLASH Companion '26)*. Association for Computing Machinery, New York, NY, USA, 18–20. doi:10.1145/3837729.3842148

[5] Karuna Grewal, Brighten Godfrey, and Justin Hsu. 2025. SafeTree: Expressive Tree Policies for Microservices. *Proc. ACM Program. Lang.* 9, OOPSLA2, Article 349 (Oct. 2025), 27 pages. doi:10.1145/3763127

[6] Karuna Grewal, Philip Brighten Godfrey, and Justin Hsu. 2023. Expressive Policies For Microservice Networks. In *Proceedings of the 22nd ACM Workshop on Hot Topics in Networks, HotNets 2023, Cambridge, MA, USA, November 28-29, 2023*. ACM, 280–286. doi:10.1145/3626111.3628181

[7] Karuna Grewal, P. Brighten Godfrey, and Justin Hsu. 2026. SafeNom: Data-Aware Microservice Policies. arXiv:2609.30394 [cs.PL] https://arxiv.org/abs/2609.30394

[8] Karuna Grewal, P. Brighten Godfrey, Justin Hsu, and Umang Mathur. 2026. SafePar: Monitoring Asynchrony in Microservices. arXiv:2609.33081 [cs.PL] https://arxiv.org/abs/2609.33081

[9] Dongyun Jin, Patrick O'Neil Meredith, Choonghwan Lee, and Grigore Roşu. 2012. JavaMOP: efficient parametric runtime monitoring framework. In *Proceedings of the 34th International Conference on Software Engineering* (Zurich, Switzerland) *(ICSE '12)*. IEEE Press, 1427–1430.

[10] Michael Kaminski and Nissim Francez. 1994. Finite-Memory Automata. *Theor. Comput. Sci.* 134, 2 (1994), 329–363. doi:10.1016/0304-3975(94)90242-9

[11] Menna Mostafa and Borzoo Bonakdarpour. 2015. Decentralized Runtime Verification of LTL Specifications in Distributed Systems. In *2015 IEEE International Parallel and Distributed Processing Symposium, IPDPS 2015, Hyderabad, India, May 25-29, 2015*. IEEE Computer Society, 494–503. doi:10.1109/IPDPS.2015.95